



DELTA-транзит

Руководство системного администратора

Версия дистрибутива: v3.4

Установщик: `delta-transit-install.sh` v3.1.0

Применённые патчи: PROMPT 01–09, PROMPT V2.0, PROMPT 10–13, PROMPT 35, PROMPT 36

Статус PROMPT 14: Не применён (см. Раздел 9)

1. Архитектура и файловая структура

DELTA-транзит — корпоративный почтовый прокси-шлюз, передающий файлы между сотрудниками офиса (референтами) и внешними контрагентами через электронную почту. Система состоит из четырёх слоёв:

- **Nginx + PHP-FPM** — веб-панель администратора (`/var/www/mail-proxy`);
- **MariaDB** — база данных `mail_proxy` (референты, аккаунты, OAuth2-токены, провайдеры);
- **iRedMail (Postfix + Dovecot)** — почтовый стек, локальная доставка референтам;
- **Python-демон** (systemd-служба `mail-proxy` , пользователь `vmail`) — мост между Maildir референтов и внешними IMAP/SMTP-серверами контрагентов.

1.1. Карта файлов дистрибутива

```
./
├─ delta-transit-install.sh      # Полный установщик (12 фаз, см. раздел 2)
├─ mail-proxy-setup.sh          # Облегчённый установщик зависимостей демона
├─ configure_limits.sh          # Настройка лимитов 150 МБ (раздел 4)
├─ mail-proxy-daemon.py         # Python-демон (systemd-служба)
├─ mail-proxy.service           # systemd unit-файл демона
├─ logrotate-mail-proxy         # Конфигурация ротации логов
├─ schema.sql                   # DDL базы данных mail_proxy
├─ requirements.txt              # Python-зависимости демона
├─ test_large_attachment.py     # Скрипт ручного теста передачи крупных вложений
├─ web/
│   ├── config.php              # APP_BASE_URL и связанные константы
│   ├── index.php               # Точка входа веб-панели, роутинг по action=
│   ├── monitor.php             # Страница мониторинга (раздел 6)
│   └─ includes/
│       ├── Cryptor.php         # AES-256-GCM/CBC-legacy шифрование
│       ├── helpers.php         # getClientIp, CSRF, SSRF-guard, БД-хелперы
│       ├── oauth2.php          # OAuth2 authorization code flow
│       └─ providers_ui.php     # CRUD-интерфейс OAuth2-провайдеров
└─ docs/                       # Технические задания и анкорный документ проекта
```

Примечание о статусе файлов репозитория.

В корне репозитория присутствуют два установочных скрипта: `delta-transit-install.sh` (полный установщик, 12 фаз, формирует production-отчёт и проводит финальный аудит) и `mail-proxy-setup.sh` (более ранний, облегчённый скрипт, который устанавливает только Python-зависимости демона, базовые группы/права и systemd-юнит, без настройки Nginx/PHP/MariaDB и без финального аудита). Оба файла рабочие и непротиворечивы друг другу по правам и группам, но на момент написания документа `mail-proxy-setup.sh` не используется основным сценарием установки — он оставлен в дистрибутиве как вспомогательный инструмент для ручной пересборки окружения демона (например, при восстановлении после сбоя без полной переустановки веб-панели). Используйте `delta-transit-install.sh` как основной установщик; обращайтесь к `mail-proxy-setup.sh` только при сценариях из раздела 10.

2. Установка и первичная настройка

2.1. Предварительные требования

Установщик `delta-transit-install.sh` ожидает следующие пакеты (устанавливаются автоматически на фазе Packages, если отсутствуют): `python3`, `python3-pip`, `python3-venv`, `mariadb-server`, `nginx`, `postfix`, `dovecot-imapd`, `logrotate`, `openssl`, `curl`. iRedMail (Postfix/Dovecot) должен быть развёрнут и настроен для доставки почты референтам до запуска установщика — установщик не разворачивает почтовый стек, только интегрируется с ним.

Обязательные файлы дистрибутива, без которых установщик завершится с ошибкой: `mail-proxy-daemon.py`, `mail-proxy.service`, `logrotate-mail-proxy`, `schema.sql`, `requirements.txt`. Запускайте установщик из директории, где лежат эти файлы.

2.2. Применение schema.sql

Установщик применяет схему автоматически на фазе Database. Если требуется применить её вручную (например, при восстановлении):

```
mysql -u root -p < schema.sql
```

Схема создаёт базу `mail_proxy` (charset `utf8mb4`, collation `utf8mb4_unicode_ci`) и пять таблиц: `referents`, `clients`, `external_accounts`, `oauth_tokens`, `oauth_providers`. Поля `auth_type`, `imap_encryption`, `smtp_encryption` в `external_accounts` защищены на уровне СУБД через `ENUM('plain', 'oauth2')` и `ENUM('none', 'ssl', 'tls')` соответственно — недопустимые значения СУБД отклонит на уровне `INSERT / UPDATE` (подробнее — раздел 9, пункт 2). Таблица `oauth_providers` предзаполняется записями для Google, Yandex и Microsoft с `ON DUPLICATE KEY`

UPDATE, то есть повторное применение `schema.sql` безопасно обновит endpoints провайдеров без создания дублей.

2.3. Запуск delta-transit-install.sh

```
sudo ./delta-transit-install.sh
```

Установщик выполняется в 12 последовательных фазах:

1. **Preflight** — проверка обязательных файлов, прав запуска (root), ОС.
2. **Packages** — установка APT-пакетов.
3. **Database** — применение `schema.sql`, создание пользователя БД.
4. **Users/Groups** — создание `mail-proxy-logs`, `mail-proxy-crypto`, удаление `www-data` из группы `vmail`, если он там оказался (см. раздел 8).
5. **Python** — создание venv в `/opt/delta-transit/venv`, установка `requirements.txt`.
6. **Web** — копирование `web/` в `/var/www/mail-proxy`, права `0644` / `0755`.
7. **Postfix** — интеграция с локальным Postfix.
8. **Dovecot** — интеграция с локальным Dovecot.
9. **PHP** — настройка `php.ini` (лимиты, раздел 4); фаза пропускается с предупреждением, если PHP не установлен.
10. **Nginx** — установка фрагмента лимита тела запроса (раздел 2.4).
11. **systemd** — установка `mail-proxy.service`, `systemd-analyze verify`, `daemon-reload`, `enable`.
12. **Validation** — запуск службы, проверка её активности, наличия лога, входа в БД, отсутствия плейсхолдеров `CHANGE_ME` / `mail-proxy.local` в конфигурации.
13. **Production audit** — финальная проверка (раздел 2.6) и запись отчёта `/root/delta-transit-install-report.txt` (права `0600`).

2.4. Настройка /etc/mail-proxy/db.conf и crypto.key

Оба файла создаются установщиком в `/etc/mail-proxy/` с правами `0640`, владелец `root:mail-proxy-crypto`. Группа `mail-proxy-crypto` создаётся установщиком и в неё включаются `www-data` (для веб-панели) и `vmail` (для демона) — это единственный легитимный путь чтения секретов обоими процессами без предоставления им избыточных прав друг на друга.

`db.conf` — INI-файл с секцией `[db]`:

```
[db]
db_host = 127.0.0.1
db_user = mail_proxy
db_pass = <пароль>
db_name = mail_proxy
```

`crypto.key` — файл из **ровно 64 шестнадцатеричных символов** (256 бит), генерируется установщиком через Python. Из этого hex-ключа оба компонента (Python-демон и PHP-Cryptor) детерминированно выводят реальный 256-битный ключ AES через `SHA-256(hex_string)`. И PHP-, и Python-сторона при чтении `crypto.key` строго проверяют длину (64 символа) и корректность hex.

2.5. APP_BASE_URL в config.php

`web/config.php` определяет константу `APP_BASE_URL`, используемую исключительно для построения `redirect_uri` в OAuth2-flow (раздел 5). **Эта константа не строится из заголовка `Host` входящего запроса** — это намеренное решение безопасности для защиты от Host header injection.

2.6. Пример конфигурации Nginx (allow/deny, X-Real-IP)

Важно:

Фаза Nginx установщика создаёт только файл `/etc/nginx/conf.d/mail-proxy.conf` со строкой `client_max_body_size 210M;` и требует **вручную** добавить `include /etc/nginx/conf.d/mail-proxy.conf;` внутрь блока `http {}`.

```

server {
    listen 443 ssl;
    server_name mail-proxy.example.local;
    root /var/www/mail-proxy;
    index index.php;

    location / {
        allow 10.0.0.0/8;
        allow 172.16.0.0/12;
        allow 192.168.0.0/16;
        allow 127.0.0.1;
        deny all;

        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param HTTP_X_REAL_IP $remote_addr;
        fastcgi_param HTTP_X_FORWARDED_FOR $proxy_add_x_forwarded_for;
        fastcgi_pass unix:/run/php/php8.1-fpm.sock;
    }
}

```

3. Управление службой

3.1. Базовые команды

```

# Старт / остановка / перезапуск
sudo systemctl start mail-proxy.service
sudo systemctl stop mail-proxy.service
sudo systemctl restart mail-proxy.service

# Текущий статус
sudo systemctl status mail-proxy.service

```

3.2. Сигналы, понимаемые демоном

Сигнал	Обработчик	Эффект
SIGTERM	_handle_signal	Иниципирует graceful shutdown (_stop_event.set())
SIGINT	_handle_signal	То же самое (Ctrl+C при запуске в foreground)
SIGUSR1	_handle_log_reopen	Переоткрывает файловые хендлеры логгера после ротации

4. Настройка лимитов вложений (150 МБ)

Целевой лимит вложения для конечного пользователя — 150 МБ. Фактический лимит на уровне почтовых протоколов и веб-сервера установлен с запасом — ≈200–210 МБ (с учётом Base64 overhead ~33%).

Скрипт `configure_limits.sh` применяет согласованные значения и перезапускает затронутые службы. Перед изменением конфигурационных файлов создаёт резервные копии с суффиксом `.bak_YYYYMMDD_....`

```
sudo ./configure_limits.sh
```

Компонент	Параметр	Значение	Назначение
Postfix	message_size_limit	209715200 (200 МБ)	Максимальный размер письма
Postfix	mailbox_size_limit	314572800 (300 МБ)	Максимальный размер ящика
MariaDB	max_allowed_packet	256M	Макс. размер пакета для крупных данных
Nginx	client_max_body_size	210M	Лимит тела HTTP-запроса веб-панели
PHP-FPM	upload_max_filesize	200M	Лимит загружаемого файла
PHP-FPM	memory_limit	512M	Лимит памяти PHP-процесса

4.1. Интерактивный шаг: квоты почтовых ящиков (MariaDB)

После настройки `max_allowed_packet` в конфигурации MariaDB скрипт останавливается и запрашивает данные с клавиатуры. Без ответов на запросы `configure_limits.sh` не продолжит настройку Nginx/PHP-FPM и перезапуск служб.

Запрос 1: пароль root MariaDB

```
MySQL root password:
```

- Ввод скрыт (символы не отображаются на экране).
- Используется тот же пароль root, который задавался при первоначальной установке MariaDB / iRedMail.
- Пароль нужен для выполнения SQL-запроса `UPDATE vmail.mailbox SET quota = ...` от имени `mysql -u root`.

Запрос 2: выбор почтовых ящиков

```
Email-адреса через запятую или --all-referents:
```

Допустимы два варианта:

1. Список адресов через запятую — например, `referent1@example.com, referent2@example.com`. Скрипт выполняет `UPDATE vmail.mailbox SET quota = 10240 WHERE username IN ('...')`;
2. Флаг `--all-referents` — скрипт задаёт дополнительный запрос Домен: и выполняет `UPDATE vmail.mailbox SET quota = 10240 WHERE username LIKE '%@<домен>';`, поднимая квоту всем ящикам указанного домена.

Значение квоты

Скрипт устанавливает `quota = 10240` в таблице `vmail.mailbox` (схема `iRedMail`). В этой колонке значение хранится в мегабайтах (МБ); `10240` соответствует 10 ГБ дисковой квоты на ящик. Это отдельный параметр от лимитов `Postfix/Nginx/PHP` в таблице выше. После выполнения запроса скрипт выводит строку вида `Updated mailbox rows: N`. Если `N = 0`, проверьте правильность адресов или домена.

Интерактивный режим — не для автоматизации. У `configure_limits.sh` нет флагов для передачи пароля `MariaDB` или списка ящиков неинтерактивно. Перенаправление `stdin` и запуск из `cron/CI` не поддерживаются — оператор должен присутствовать у консоли и отвечать на запросы вручную.

6. Мониторинг

Страница `monitor.php` доступна по адресу `https://<APP_BASE_URL>/monitor.php` и отображает текущее состояние службы, доступность логов, критические ошибки и статус авторизации OAuth2/IMAP/SMTP.

7. Резервное копирование и восстановление

Критически важные объекты для бэкапа:

- Ключ шифрования: `/etc/mail-proxy/crypto.key` (Максимальная критичность)
- Конфигурация БД: `/etc/mail-proxy/db.conf`
- Дамп базы данных MariaDB `mail_proxy`

8. Контрольный список безопасности

№	Пункт	Статус	Подтверждение в коде
1	<code>www-data</code> не входит в группу <code>vmail</code>	✓ Закрит	Активно удаляется во время фазы Users/Groups.
2	Права на <code>crypto.key</code> / <code>db.conf</code>	✓ Закрит	Установлены права <code>0640</code> (root:mail-proxy-crypto).
3	systemd hardening	✓ Закрит	Unit включает NoNewPrivileges, ProtectSystem=strict и др.
4	SSRF-защита (OAuth2 эндпоинты)	✓ Закрит	<code>assertSafeOAuthEndpoint()</code> блокирует loopback/приватные IP.

9. Известные ограничения и эксплуатационные особенности

9.1. Буферизация писем в памяти при приёме через IMAP. Библиотека `imaplib` возвращает письмо целиком в памяти процесса. При пиковой нагрузке формула расхода RAM:

$$\text{RAM_peak} \approx \text{IMAP_WORKER_COUNT} \times \text{размер_письма_RFC822} \approx 20 \times 200 \text{ МБ} \approx 4 \text{ ГБ}$$

9.3. Разрыв dot-stuffing на границе SMTP-чанков. Статус: закрыт (PROMPT 10). Функция удерживает последний байт каждого чанка в переменной `carry` для исключения разрывов экранирования на стыках.

10. Типовые неисправности (troubleshooting)

При ошибках старта проверьте валидность hex-ключа: `wc -c /etc/mail-proxy/crypto.key` (ожидается 64 символа) и права доступа.

11. Обновление системы

Штатный перезапуск службы безопасен в любой момент. Входящие письма остаются в статусе `UNSEEN` до подтверждения локальной доставки, а исходящие сохраняются в `Maildir/new` референтов.